BRAC University — Department of Computer Science & Engineering
CSE 221: Algorithms | Mid Semester Exam | Summer 2022
Complete Solution Guide (Set A & Set B)

SET A

Question 1(a) — Time Complexity of worstCase(n) [Set A]

Code:

```
for ( i = 0; i < n; i = i + 3 ) ← outer loop
for ( j = n; j >= 1; j = j / 5 ) ← middle loop
for ( k = 1; k <= n; k = k * 5 ) ← inner loop
sum = a + b ← O(1) work
```

Analysis of each loop:

- **Outer loop (i):** i goes 0, 3, 6, …, n–1 → iterates ■**n/3**■ **= O(n)** times.
- **Middle loop (j):** j goes n, n/5, n/25, … down to 1. Number of steps = $\log_5(n)$ = **O(log n)**.
- **Inner loop (k):** k goes 1, 5, 25, … up to n. Number of steps = $\log_5(n)$ = **O(log n)**.

Total time complexity:

T(n) = O(n) × O(log n) × O(log n) = **O(n $\log^2 n$)**

Question 1(b) — Recurrence Relation [Set A]

Recurrence: **T(n) = 2T(n/4) + n**

Using Master Theorem: T(n) = aT(n/b) + f(n)

- a = 2, b = 4, f(n) = n
- $\log_b a = \log_4 2 =$ **1/2** → $n^{\log_b a} = n^{0.5} = \sqrt{n}$
- Compare f(n) = n with $n^{0.5}$:

  f(n) = $n = n^{0.5} \cdot n^{0.5}$ → $f(n) = \Omega(n^{\log_b a + \varepsilon})$ with $\varepsilon = 0.5 > 0$

  - **Check regularity:** $2 \cdot f(n/4) = 2 \cdot (n/4) = n/2 \leq c \cdot n$ for $c = 1/2 < 1$ ✓
  - This matches **Master Theorem Case 3**.

∴ **T(n) = Θ(n)**

Question 2(a) — Sorting Algorithm [Set A]

Algorithm chosen: Heap Sort

Reason: The problem requires sorting in descending order, is space-efficient (in-place, O(1) extra space — solving the space concern), and does not depend on the initial order (patient about O(n log n) time always).

Initial array (Views in thousands):

Index	0	1	2	3	4	5	6	7
Track	Harridan	Of The New Day	Rats Return	Dignity	Herd Culling	Walk the Plank	Chimera's Wreck	Population Three
Views	15	8	11	112	33	39	88	41

Step 1: Build Max-Heap (Heapify from last non-leaf → index 3)

```
Array: [15, 8, 11, 112, 33, 39, 88, 41] (0-indexed)
```

Heapify index 3 (112): children are index 7 (41) → 112 is already largest.

Heapify index 2 (11): children are index 5 (39), index 6 (88) → swap 11 with 88 → [15, 8, 88, 112, 33, 39, 11, 41]

Heapify index 1 (8): children are index 3 (112), index 4 (33) → swap 8 with 112 → [15, 112, 88, 8, 33, 39, 11, 41]

→ Heapify index 3 (8): children are index 7 (41) → swap 8 with 41 → [15, 112, 88, 41, 33, 39, 11, 8]

Heapify index 0 (15): children are index 1 (112), index 2 (88) → swap 15 with 112 → [112, 15, 88, 41, 33, 39, 11, 8]

→ Heapify index 1 (15): children are index 3 (41), index 4 (33) → swap 15 with 41 → [112, 41, 88, 15, 33, 39, 11, 8]

→ Heapify index 3 (15): children are index 7 (8) → 15 is largest. Stop.

Max-Heap built: [112, 41, 88, 15, 33, 39, 11, 8]

Step 2: Extract max repeatedly and build sorted (ascending) array, then reverse for descending

We want descending order (highest first). Heap sort naturally produces ascending; we can either use a min-heap or reverse. Here we show extracting and tracking:

Step	Action	Heap After Heapify
1	Extract 112	Swap 112↔8 → [8,41,88,15,33,39,11 | 112] → heapify → [88,41,39,15,33,8,11]
2	Extract 88	Swap 88↔11 → [11,41,39,15,33,8 | 88] → heapify → [41,33,39,15,8,11]
3	Extract 41	Swap 41↔11 → [11,33,39,15,8 | 41] → heapify → [39,33,11,15,8]
4	Extract 39	Swap 39↔8 → [8,33,11,15 | 39] → heapify → [33,15,11,8]
5	Extract 33	Swap 33↔8 → [8,15,11 | 33] → heapify → [15,8,11]
6	Extract 15	Swap 15↔11 → [11,8 | 15] → heapify → [11,8]
7	Extract 11	Swap 11↔8 → [8 | 11] → trivial
8	Extract 8	Last element

Final sorted order (highest → lowest views):

Listen Order	Track	Views (thousands)
1	Dignity	112
2	Chimera's Wreck	88
3	Population Three	41
4	Walk the Plank	39
5	Herd Culling	33
6	Harridan	15
7	Rats Return	11
8	Of The New Day	8

Question 2(b) — Re-running Heap Sort on Sorted List [Set A]

When Heap Sort is run on an already-sorted (descending) list:

- **Build-Heap phase:** Still takes O(n) — same for any input.
- **Extraction phase:** Each of the n extractions calls heapify (O(log n)), regardless of initial order.
- Heap Sort has **no best-case advantage**; it is always Θ(n log n).

∴ **Run-time complexity = Θ(n log n)**

Question 3 — Graph & DFS [Set A]

The graph nodes (from the image) represent parts of the arithmetic expression **(6 ÷ 2) + 5 = 8**.

Nodes visible in the graph: **'(' , 6 , ÷ , 2 , ')' , + , 5 , = , 8** (9 nodes total)

Question 3(a) — Number of edges in the DFS tree

A DFS tree on a connected graph of **V vertices** always has exactly **V** – **1 edges**.

The expression tree for (6 ÷ 2) + 5 = 8 has **9 nodes**.

∴ **Number of edges in DFS tree = 9** – **1 = 8**

Question 3(b) — Is the equation achievable by DFS from '('?

We need to verify that starting DFS from '(' can produce a traversal spelling out: **(6 ÷ 2) + 5 = 8**.

DFS traversal simulation (selecting neighbors to match the equation):

Step	Node Visited	Action / Stack State
1	Visit '('	Start node. Push '(' to stack. Discovered: {(}
2	Visit '6'	Neighbor of '(' → visit 6. Stack: (, 6
3	Visit '÷'	Neighbor of '6' → visit ÷. Stack: (, 6, ÷
4	Visit '2'	Neighbor of '÷' → visit 2. Stack: (, 6, ÷, 2
5	Backtrack to '÷','6','('	No unvisited neighbors from 2 → backtrack up to '('
6	Visit ')'	Next unvisited neighbor of '(' → visit ')'. Stack: (,)
7	Visit '+'	Neighbor of ')' → visit +. Stack: (,), +
8	Visit '5'	Neighbor of '+' → visit 5. Stack: (,), +, 5
9	Visit '='	Neighbor of '+' (or 5) → visit =. Stack: ..., =
10	Visit '8'	Neighbor of '=' → visit 8. DFS complete.

Conclusion: Yes, the equation **(6 ÷ 2) + 5 = 8** is achievable by running DFS from '(' by carefully choosing the neighbor/edge selection order that follows the expression structure. **Bill is correct.**

Question 3(c) — Edge Classification in DFS from '(' [Set A]

In a DFS on a directed/undirected graph, edges are classified as:

- **Tree Edge:** Edge used to discover a new (white) node during DFS.

- **Back Edge:** Edge from a node to one of its *ancestors* in the DFS tree.
- **Forward Edge:** Edge from a node to a *descendant* (non-tree path) in the DFS tree.

From the graph (9 nodes, edges connecting expression components):

Based on the DFS traversal from '(':

Edge	Classification	Reason
(→ 6	Tree Edge	6 first discovered from (
6 → ÷	Tree Edge	÷ first discovered from 6
÷ → 2	Tree Edge	2 first discovered from ÷
(→)	Tree Edge	) first discovered from ((after backtrack)
(→ +	Tree Edge	+ first discovered from) or (
+ → 5	Tree Edge	5 first discovered from +
+ → =	Tree Edge	= first discovered from +
= → 8	Tree Edge	8 first discovered from =
2 → (or 6 → (	Back Edge	Points back to ancestor '('
5 → (or others	Forward Edge	Points to a descendant already visited

Note: The exact back/forward edge set depends on the precise edge list in the graph image. Tree edges form the primary DFS path; any edge leading to an already-visited ancestor is a back edge; any edge skipping ahead to a visited descendant is a forward edge.

SET B

Question 1(a) — Time Complexity of worstCase(n) [Set B]

Code:

```
for ( i = 1; i < n; i = i + 3 ) ← outer loop
for ( j = n; j >= 1; j = j - 2 ) ← middle loop
for ( k = n; k >= 1; k = k / 4 ) ← inner loop
sum = a + b ← O(1) work
```

Analysis of each loop:

- **Outer loop (i):** i goes 1, 4, 7, …, up to n → iterates ■(n–1)/3■ = **O(n)** times.
- **Middle loop (j):** j goes n, n–2, n–4, … down to 1 → iterates ■n/2■ = **O(n)** times.
- **Inner loop (k):** k goes n, n/4, n/16, … down to 1 → iterates $\log_4(n)$ = **O(log n)** times.

Total time complexity:

$T(n) = O(n) \times O(n) \times O(\log n) =$ **$O(n^2 \log n)$**

Question 1(b) — Recurrence Relation [Set B]

Recurrence: **$T(n) = 2T(n/2) + 1/n$**

Using Master Theorem: $T(n) = aT(n/b) + f(n)$

- $a = 2$, $b = 2$, $f(n) = 1/n = n^{-1}$
- $\log_b a = \log_2 2 = 1 \rightarrow n^{\log_b a} = n^1 = n$
- Compare $f(n) = n^{-1}$ with n^1:

  $f(n) = n^{-1} = O(n^{1-\varepsilon})$ for $\varepsilon = 2$ (since n^{-1} grows far slower than n)

  - This matches **Master Theorem Case 1**: $f(n) = O(n^{\log_b a - \varepsilon})$.

∴ **$T(n) = \Theta(n \log n)$ [since a = b in Case 1 → $\Theta(n^{\log_b a}) = \Theta(n)$]**

More precisely: $T(n) = \Theta(n)$

Question 2(a) — Sorting Algorithm [Set B]

Algorithm chosen: Heap Sort (same reasoning as Set A — in-place, O(1) space, O(n log n) always)

Initial array (Views in thousands):

Index	0	1	2	3	4	5	6	7
Track	Harridan	Of The New Day	Rats Return	Dignity	Herd Culling	Walk the Plank	Chimera's Wreck	Population Three
Views	17	10	15	88	35	41	90	38

Step 1: Build Max-Heap

```
Array: [17, 10, 15, 88, 35, 41, 90, 38] (0-indexed)
```

Last non-leaf index = ⌊8/2⌋ – 1 = 3

Heapify index 3 (88): children → index 7 (38). 88 > 38, no swap.

Heapify index 2 (15): children → index 5 (41), index 6 (90). 90 is largest → swap 15 with 90 → [17,10,90,88,35,41,15,38]

Heapify index 1 (10): children → index 3 (88), index 4 (35). 88 is largest → swap 10 with 88 → [17,88,90,10,35,41,15,38]

→ Heapify index 3 (10): children → index 7 (38). Swap → [17,88,90,38,35,41,15,10]

Heapify index 0 (17): children → index 1 (88), index 2 (90). 90 is largest → swap 17 with 90 → [90,88,17,38,35,41,15,10]

→ Heapify index 2 (17): children → index 5 (41), index 6 (15). 41 is largest → swap → [90,88,41,38,35,17,15,10]

Max-Heap built: [90, 88, 41, 38, 35, 17, 15, 10]

Step 2: Sort by extracting max

Step	Action	Result
1	Extract 90	Swap 90↔10 → heapify → [88,38,41,10,35,17,15 | 90]
2	Extract 88	Swap 88↔15 → heapify → [41,38,17,10,35,15 | 88]
3	Extract 41	Swap 41↔15 → heapify → [38,35,17,10,15 | 41]
4	Extract 38	Swap 38↔15 → heapify → [35,15,17,10 | 38]
5	Extract 35	Swap 35↔10 → heapify → [17,15,10 | 35]
6	Extract 17	Swap 17↔10 → heapify → [15,10 | 17]
7	Extract 15	Swap 15↔10 → [10 | 15]
8	Extract 10	Last element

Final sorted order (highest → lowest views):

Listen Order	Track	Views (thousands)
1	Chimera's Wreck	90
2	Dignity	88
3	Walk the Plank	41
4	Population Three	38
5	Herd Culling	35
6	Harridan	17
7	Rats Return	15
8	Of The New Day	10

Question 2(b) — Re-running Heap Sort on Sorted List [Set B]

Same analysis as Set A — Heap Sort's runtime is independent of input order.

- Build-Heap: O(n) always.
- n extractions each O(log n): O(n log n).

∴ **Run-time complexity = Θ(n log n) (no best-case improvement)**

Question 3 — BFS on Graph [Set B]

Graph Setup: Nodes labeled 3 to 12 (10 nodes). Given edges: (5,6),(5,10),(6,11),(7,11).

Rule a: All multiples of 3 in range → {3, 6, 9, 12} → edges: (3,6),(3,9),(3,12),(6,9),(6,12),(9,12)

Rule b: All multiples of 4 in range → {4, 8, 12} → edges: (4,8),(4,12),(8,12)

Complete Edge List:

Given: (5,6),(5,10),(6,11),(7,11)

Multiples of 3: (3,6),(3,9),(3,12),(6,9),(6,12),(9,12)

Multiples of 4: (4,8),(4,12),(8,12)

Question 3(a) — Draw the Graph

The graph has 10 nodes (3–12) and the following edges:

```
(3,6),(3,9),(3,12),(4,8),(4,12),(5,6),(5,10),(6,9),(6,11),(6,12),(7,11),(8,12),(9,12)
```

Question 3(b) — At Least Four Triangles?

A triangle is 3 nodes all mutually connected. Let's identify them:

	Triangle Nodes	Edges Present
Triangle 1	3 — 6 — 9	Edges: (3,6),(6,9),(3,9) ✓
Triangle 2	3 — 6 — 12	Edges: (3,6),(6,12),(3,12) ✓
Triangle 3	3 — 9 — 12	Edges: (3,9),(9,12),(3,12) ✓
Triangle 4	6 — 9 — 12	Edges: (6,9),(9,12),(6,12) ✓
Triangle 5	4 — 8 — 12	Edges: (4,8),(8,12),(4,12) ✓
Triangle 6	3 — 6 — 12 (& 9)	The 4-clique {3,6,9,12} contains C(4,3)=4 triangles

Conclusion: Yes, Bill is correct. There are at least **5 triangles** (4 from the {3,6,9,12} clique + 1 from {4,8,12}), which is more than four.

Question 3(c) — BFS from Node 3 (Shortest distances to all nodes)

Running BFS starting from Node 3:

Adjacency list (from complete edge list):

```
Node 3: [6, 9, 12]
Node 4: [8, 12]
Node 5: [6, 10]
Node 6: [3, 5, 9, 11, 12]
Node 7: [11]
Node 8: [4, 12]
Node 9: [3, 6, 12]
Node 10: [5]
Node 11: [6, 7]
Node 12: [3, 4, 6, 8, 9]
```

BFS execution (Queue-based):

BFS Step	Action	Distances Updated	Queue After
Init	Queue: [3]	Dist: {3:0}	Visited: {3}
Process 3	Neighbors: 6,9,12	Dist: {3:0, 6:1, 9:1, 12:1}	Queue: [6,9,12]
Process 6	Neighbors: 3✓,5,9✓,11,12✓ → new: 5,11	Dist: {5:2, 11:2}	Queue: [9,12,5,11]
Process 9	Neighbors: 3✓,6✓,12✓ → none new	—	Queue: [12,5,11]
Process 12	Neighbors: 3✓,4,6✓,8,9✓ → new: 4,8	Dist: {4:2, 8:2}	Queue: [5,11,4,8]
Process 5	Neighbors: 6✓,10 → new: 10	Dist: {10:3}	Queue: [11,4,8,10]
Process 11	Neighbors: 6✓,7 → new: 7	Dist: {7:3}	Queue: [4,8,10,7]
Process 4,8,10,7	All remaining neighbors already visited	—	Queue: []

Final Shortest Distances from Node 3:

From Node 3	To Node	Shortest Distance
3	3	0
3	6	1
3	9	1
3	12	1
3	4	2
3	5	2
3	8	2
3	11	2
3	7	3
3	10	3

Question 3(d) — Maximum Additional Edges

A simple undirected graph on n = **10** nodes can have at most **C(10,2) = 10×9/2 = 45** edges.

Current number of edges in the graph: **13**

Maximum possible edges: 45

∴ **Additional edges that can be added = 45 – 13 = 32**